Dynamic Memory Allocation in C

Dynamic Memory Allocation is an important topic in C programming because it allows a program to **allocate and release memory during program execution**.

The main library functions used for dynamic memory allocation are:

1. `malloc()`
2. `calloc()`
3. `realloc()`
4. `free()`

These functions are declared in the **`<stdlib.h>`** header file.

In C, memory can be allocated in two general ways:

Static/Fixed Memory Allocation

The size of memory is decided before or at compile time.

```
int arr[10];
```

Here, the array can store exactly 10 integers.

Dynamic Memory Allocation

Memory is allocated **at run time**, according to the program's requirements.

For example:

```
int *ptr;

ptr = malloc(10 * sizeof(int));
```

Here, memory for 10 integers is allocated during program execution.

Dynamic memory allocation is the process of allocating and managing memory at run time according to the requirements of a program.

Dynamic memory is generally obtained from the **heap**.

Why Dynamic Memory Allocation is Needed

Dynamic memory allocation is useful when the required amount of memory is **not known in advance**.

Example

Suppose a program needs to store marks of students, but the number of students will be entered by the user.

```
int n;

printf("Enter number of students: ");
scanf("%d", &n);
```

We can then allocate memory according to `n`:

```
int *marks;

marks = malloc(n * sizeof(int));
```

Thus, the program does not need to reserve a fixed amount of memory beforehand.

Advantages

1. Memory is allocated according to actual requirements.
2. Memory can be increased or decreased during execution.
3. It helps avoid unnecessary memory allocation.
4. It is useful for dynamic data structures such as:
    - Linked lists
    - Stacks
    - Queues
    - Trees
    - Graphs

Header File

The dynamic memory allocation functions are provided by:

```
#include <stdlib.h>
```

The four important functions are:

```
malloc()
calloc()
realloc()
free()
```

`1. malloc()` Function

`malloc` stands for **Memory Allocation**.

It allocates a specified number of **bytes of uninitialized memory**.

Syntax

```
ptr = (data_type *) malloc(number_of_bytes);
```

In C, the cast is generally unnecessary:

```
ptr = malloc(number_of_bytes);
```

Example

```
int *ptr;

ptr = malloc(5 * sizeof(int));
```

This allocates enough memory to store **5 integers**.

Example Program Using `malloc()`

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr;
    int n = 5;

    ptr = malloc(n * sizeof(int));

    if (ptr == NULL)
    {
        printf("Memory allocation failed");
        return 1;
    }

    for (int i = 0; i < n; i++)
    {
        ptr[i] = (i + 1) * 10;
    }

    for (int i = 0; i < n; i++)
    {
        printf("%d ", ptr[i]);
    }

    free(ptr);

    return 0;
}
```

Output

```
10 20 30 40 50
```

Important Point About `malloc()`

The memory allocated by `malloc()` contains **indeterminate values** initially.

For example:

```
int *p = malloc(5 * sizeof(int));
```

The allocated integers do **not automatically become zero**.

Therefore, initialize them before using their values.

2. `calloc()` Function

`calloc` stands for **Contiguous Allocation**.

It allocates memory for multiple elements and initializes all allocated bytes to **zero**.

Syntax

```
ptr = calloc(number_of_elements, size_of_each_element);
```

Example

```
int *ptr;

ptr = calloc(5, sizeof(int));
```

This allocates memory for 5 integers and initializes the allocated storage to zero.

Example Program Using `calloc()`

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr;
    int n = 5;

    ptr = calloc(n, sizeof(int));

    if (ptr == NULL)
    {
        printf("Memory allocation failed");
        return 1;
    }

    for (int i = 0; i < n; i++)
    {
        printf("%d ", ptr[i]);
    }

    free(ptr);

    return 0;
}
```

Output

```
0 0 0 0 0
```

3. `realloc()` Function

`realloc` stands for **Reallocation**.

It is used to **resize previously allocated memory**.

Suppose we initially allocate memory for 5 integers:

```
int *ptr;

ptr = malloc(5 * sizeof(int));
```

Later, we need space for 10 integers.

Instead of allocating an entirely new block ourselves, we can use:

```
ptr = realloc(ptr, 10 * sizeof(int));
```

Syntax

```
ptr = realloc(ptr, new_size);
```

Example Program Using `realloc()`

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr;

    ptr = malloc(5 * sizeof(int));

    if (ptr == NULL)
    {
        printf("Memory allocation failed");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        ptr[i] = i + 1;
    }

    ptr = realloc(ptr, 10 * sizeof(int));

    if (ptr == NULL)
    {
        printf("Reallocation failed");
        return 1;
    }

    for (int i = 5; i < 10; i++)
    {
        ptr[i] = i + 1;
    }

    for (int i = 0; i < 10; i++)
    {
        printf("%d ", ptr[i]);
    }

    free(ptr);

    return 0;
```

```
}
```

Output

```
1 2 3 4 5 6 7 8 9 10
```

Important Point

`realloc()` may:

- Expand the existing memory block, or
- Move the data to a new memory location if necessary.

The existing data is preserved up to the smaller of the old and new sizes.

4. `free()` Function

The `free()` function is used to **release dynamically allocated memory** when it is no longer required.

Syntax

```
free(ptr);
```

Example

```
int *ptr;

ptr = malloc(5 * sizeof(int));

/* use memory */

free(ptr);
```

After `free(ptr)`, the allocated memory is returned to the memory allocator for reuse.

Why is `free()` Important?

If dynamically allocated memory is not released after it is no longer needed, the program may suffer from a **memory leak**.

Memory Leak

A memory leak occurs when dynamically allocated memory is no longer needed but remains allocated because the program has lost the ability to release it.

Example:

```
int *p = malloc(100 * sizeof(int));

/* memory is used */

/* forgot to call free(p) */
```

The memory remains allocated until the process ends.

Complete Example Using All Four Functions

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr;
    int n = 3;

    /* malloc */
    ptr = malloc(n * sizeof(int));

    if (ptr == NULL)
    {
        printf("Memory allocation failed");
        return 1;
    }

    ptr[0] = 10;
    ptr[1] = 20;
    ptr[2] = 30;

    /* realloc */
    ptr = realloc(ptr, 5 * sizeof(int));

    if (ptr == NULL)
    {
        printf("Reallocation failed");
        return 1;
    }

    ptr[3] = 40;
    ptr[4] = 50;

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", ptr[i]);
    }

    /* free */
    free(ptr);

    return 0;
}
```

Difference Between `malloc()` and `calloc()`

This is a **very important university exam question**.

`malloc()`	`calloc()`
Allocates a single block of memory.	Allocates memory for multiple elements.
Takes **one argument**.	Takes **two arguments**.
`malloc(size)`	`calloc(n, size)`
Memory is not initialized.	Allocated bytes are initialized to zero.
Example: `malloc(5 * sizeof(int))`	Example: `calloc(5, sizeof(int))`

Difference Between `malloc()`, `calloc()`, `realloc()` and `free()`

Function	Purpose
`malloc()`	Allocates a block of uninitialized memory
`calloc()`	Allocates memory for multiple elements and initializes it to zero
`realloc()`	Changes the size of an existing allocation
`free()`	Releases dynamically allocated memory

Typical Dynamic Memory Allocation Flow

```
 Allocate Memory
       ↓
malloc() / calloc()
       ↓
   Use Memory
       ↓
Need More/Less?
       ↓
   realloc()
       ↓
 Finish Using It
       ↓
     free()
```

Introduction to File Handling

A **file** is a collection of data stored on a secondary storage device such as a hard disk or SSD.

C provides several functions for working with files. These functions are available through the **`stdio.h`** header file.

```
#include <stdio.h>
```

File handling is the process of creating, opening, reading, writing, updating, and closing files using a programming language.

Why is File Handling Required?

File handling is used when data needs to be stored **permanently**.

For example:

```
Program → Data → File → Stored permanently
```

When the program runs again, it can read the previously stored data from the file.

File Types

In C, files are commonly classified into two types:

1. **Text Files**
2. **Binary Files**

1. Text Files

A text file stores data in the form of **characters that can generally be read by humans**.

Examples:

```
students.txt
marks.txt
employees.txt
```

For example, a file may contain:

```
101 Rahul 85
102 Amit 90
103 Priya 88
```

Features

- Data is stored as characters.
- Human-readable.
- Can be opened using a text editor.
- Functions such as `fprintf()`, `fscanf()`, `fgets()`, and `fputs()` are commonly used.

2. Binary Files

A binary file stores data in **binary representation**, rather than as human-readable characters.

Examples:

```
student.dat
employee.dat
data.bin
```

Binary files are commonly used when programs need to store and retrieve data efficiently.

Features

- Data is stored in binary form.
- Usually not directly human-readable.
- Can be more compact and efficient for certain applications.
- Functions such as `fread()` and `fwrite()` are commonly used.

Difference Between Text and Binary Files

Text File	Binary File
Stores data as characters/text representation.	Stores data in binary representation.
Generally human-readable.	Generally not human-readable.
Usually larger for numeric data.	Can be more compact.
`fprintf()`, `fscanf()` etc. are commonly used.	`fread()`, `fwrite()` are commonly used.
Easy to edit using a text editor.	Usually requires a program to interpret the data.

File Operations

The major operations performed on files are:

1. **Creating a file**
2. **Opening a file**
3. **Reading from a file**
4. **Writing to a file**
5. **Appending data**
6. **Updating a file**
7. **Closing a file**

General File Handling Process

```
Open File
    ↓
Perform Read/Write Operation
    ↓
Process Data
    ↓
Close File
```

File Pointer

A **file pointer** is a pointer of type `FILE` that is used by C to identify and manage an opened file.

It is declared as:

```
FILE *fp;
```

Here:

- `FILE` → predefined type provided by `stdio.h`
- `fp` → file pointer

Example

```
#include <stdio.h>

int main()
{
    FILE *fp;

    fp = fopen("data.txt", "r");

    fclose(fp);

    return 0;
}
```

The file pointer `fp` is used by functions such as:

```
fopen()
```

```
fclose()
fprintf()
fscanf()
fgetc()
fputc()
fread()
fwrite()
```

Important Point

A file pointer does not directly contain the entire file. It refers to the stream and associated information used by the C library to perform operations on the opened file.

File Opening Modes

A file is opened using the `fopen()` function.

Syntax

```
FILE *fp;

fp = fopen("filename", "mode");
```

The **mode** specifies what operation will be performed on the file.

1. "r" — Read Mode

Opens an existing file for reading.

```
fp = fopen("data.txt", "r");
```

If the file does not exist, opening fails.

2. "w" — Write Mode

Opens a file for writing.

```
fp = fopen("data.txt", "w");
```

If the file already exists, its previous contents are **discarded**.

If it does not exist, a new file is created.

3. "a" — Append Mode

Opens a file for adding data at the end.

```
fp = fopen("data.txt", "a");
```

Existing contents are preserved, and new data is written at the end.

If the file does not exist, it is created.

4. "r+" — Read and Write

Opens an existing file for both reading and writing.

```
fp = fopen("data.txt", "r+");
```

The file must already exist.

5. "w+" — Write and Read

Opens a file for both writing and reading.

```
fp = fopen("data.txt", "w+");
```

If the file exists, its previous contents are discarded.

If it does not exist, a new file is created.

6. "a+" — Append and Read

Opens a file for both reading and appending.

```
fp = fopen("data.txt", "a+");
```

Existing contents are preserved, and writing takes place at the end.

Binary File Modes

The letter `b` can be added to the mode to indicate binary mode.

Examples:

```
"rb", "wb", "ab", "rb+", "wb+", "ab+"
```

For example:

```
fp = fopen("student.dat", "rb");
```

opens a binary file for reading.

File Handling Functions

Important file handling functions in C include:

Function	Purpose
`fopen()`	Opens a file
`fclose()`	Closes a file
`fgetc()`	Reads one character
`fputc()`	Writes one character
`fgets()`	Reads a string/line

`fputs()`	Writes a string
`fprintf()`	Writes formatted data
`fscanf()`	Reads formatted data
`fread()`	Reads binary/block data
`fwrite()`	Writes binary/block data
`fseek()`	Moves file position
`ftell()`	Returns current file position
`rewind()`	Moves position to beginning
`feof()`	Tests end-of-file indicator

1. fopen() Function

`fopen()` is used to open a file.

Syntax

```
FILE *fopen(const char *filename, const char *mode);
```

Example

```
FILE *fp;

fp = fopen("data.txt", "r");
```

It returns a file pointer on successful opening.

If the file cannot be opened, it returns `NULL`.

Example

```
if (fp == NULL)
{
    printf("File could not be opened");
}
```

2. fclose() Function

`fclose()` closes an opened file.

Syntax

```
fclose(fp);
```

Example

```
FILE *fp;

fp = fopen("data.txt", "r");

if (fp != NULL)
{
    /* file operations */
```

```
    fclose(fp);
}
```

Closing a file is important because it releases associated resources and ensures pending output is properly handled.

Character Input and Output

fputc()

Used to write a single character to a file.

Syntax

```
fputc(character, fp);
```

Example

```
fputc('A', fp);
```

fgetc()

Used to read a single character from a file.

Syntax

```
ch = fgetc(fp);
```

Example

```
char ch;

ch = fgetc(fp);
```

String Input and Output

fputs()

Writes a string to a file.

```
fputs("Hello World", fp);
```

fgets()

Reads a line/string from a file.

```
char str[100];

fgets(str, sizeof(str), fp);
```

Formatted File Input and Output

fprintf()

`fprintf()` is used to write formatted data to a file.

Syntax

```
fprintf(fp, "format", variables);
```

Example

```
fprintf(fp, "%d %s %.2f\n", 101, "Rahul", 85.5);
```

fscanf()

`fscanf()` is used to read formatted data from a file.

Syntax

```
fscanf(fp, "format", &variables);
```

Example

```
int roll;
float marks;

fscanf(fp, "%d %f", &roll, &marks);
```

Example: Writing to and Reading from a Text File

```
#include <stdio.h>

int main()
{
    FILE *fp;
    char name[30];

    fp = fopen("student.txt", "w");

    if (fp == NULL)
    {
        printf("Unable to open file");
        return 1;
    }

    fprintf(fp, "Rahul 85");
    fclose(fp);

    fp = fopen("student.txt", "r");

    if (fp == NULL)
    {
        printf("Unable to open file");
        return 1;
```

```
    }

    fscanf(fp, "%s", name);

    printf("Name: %s", name);

    fclose(fp);

    return 0;
}
```

File Handling Through Command Line Arguments

A C program can receive information from the command line when it is executed.

The `main()` function can be written as:

```
int main(int argc, char *argv[])
```

Here:

- `argc` → **argument count**
- `argv` → **argument vector**, an array of strings containing the arguments

Example

Suppose the program is executed as:

```
program data.txt
```

Then:

```
argc = 2
argv[0] = "program"
argv[1] = "data.txt"
```

Using Command Line Argument to Open a File

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    FILE *fp;
    char ch;

    if (argc != 2)
    {
        printf("Usage: program filename");
        return 1;
    }

    fp = fopen(argv[1], "r");

    if (fp == NULL)
    {
        printf("Unable to open file");
```

```
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF)
    {
        putchar(ch);
    }

    fclose(fp);

    return 0;
}
```

Execution

For example:

```
program data.txt
```

The filename `data.txt` is received through:

```
argv[1]
```

and used in:

```
fopen(argv[1], "r");
```

Advantages

- File names do not need to be hard-coded.
- The same program can work with different files.
- Useful for command-line utilities and batch processing.

Record I/O in Files

A **record** is a collection of related data items representing one complete entity.

For example, a student record may contain:

```
Roll Number
Name
Marks
```

In C, structures are commonly used to represent records.

Example Structure

```
struct Student
{
    int roll_no;
    char name[30];
    float marks;
};
```

One variable of this type represents one student record.

Writing Records Using fwrite()

`fwrite()` is used to write a block of data to a file.

Syntax

```
fwrite(&variable, sizeof(variable), 1, fp);
```

Example

```
struct Student s = {101, "Rahul", 85.5};

fwrite(&s, sizeof(s), 1, fp);
```

This writes the structure as a binary record.

Reading Records Using fread()

`fread()` is used to read a block of data from a file.

Syntax

```
fread(&variable, sizeof(variable), 1, fp);
```

Example

```
struct Student s;

fread(&s, sizeof(s), 1, fp);
```

The record is read into the structure variable `s`.

Complete Record I/O Example

```
#include <stdio.h>

struct Student
{
    int roll_no;
    char name[30];
    float marks;
};

int main()
{
    FILE *fp;
    struct Student s1, s2;

    s1.roll_no = 101;
    snprintf(s1.name, sizeof(s1.name), "Rahul");
    s1.marks = 85.5;

    /* Write record */
```

```
    fp = fopen("student.dat", "wb");

    if (fp == NULL)
    {
        printf("Unable to open file");
        return 1;
    }

    fwrite(&s1, sizeof(s1), 1, fp);
    fclose(fp);

    /* Read record */
    fp = fopen("student.dat", "rb");

    if (fp == NULL)
    {
        printf("Unable to open file");
        return 1;
    }

    if (fread(&s2, sizeof(s2), 1, fp) == 1)
    {
        printf("Roll No: %d\n", s2.roll_no);
        printf("Name: %s\n", s2.name);
        printf("Marks: %.2f\n", s2.marks);
    }

    fclose(fp);

    return 0;
}
```

Output

```
Roll No: 101
Name: Rahul
Marks: 85.50
```

Important File Position Functions

`fseek()`

Moves the file position indicator to a specified location.

```
fseek(fp, 0, SEEK_SET);
```

Common origins:

```
SEEK_SET → Beginning of file
SEEK_CUR → Current position
SEEK_END → End of file
```

`ftell()`

Returns the current position of the file pointer.

```
long position;
```

```
position = ftell(fp);
```

`rewind()`

Moves the file position back to the beginning.

```
rewind(fp);
```

Introduction to Graphics

Computer Graphics is the branch of computer science that deals with the creation, manipulation, storage, and display of pictures and graphical objects using a computer.

In C programming, graphics can be used to draw objects such as **lines, circles, rectangles, arcs, polygons, and other shapes** on the screen.

Traditional C graphics programming is commonly associated with the **BGI (Borland Graphics Interface)** library and the header file:

```
#include <graphics.h>
```

Applications of Computer Graphics

- Computer-Aided Design (CAD)
- Games and animation
- Graphical user interfaces (GUI)
- Educational software
- Engineering and scientific visualization
- Image processing
- Presentations and simulations

Graphics Constants

Graphics programming uses several predefined constants to specify colors, line styles, filling patterns, and other properties.

Common Color Constants

Constant	Meaning
`BLACK`	Black color
`BLUE`	Blue color
`GREEN`	Green color
`CYAN`	Cyan color
`RED`	Red color
`MAGENTA`	Magenta color
`BROWN`	Brown color
`LIGHTGRAY`	Light gray color
`DARKGRAY`	Dark gray color
`LIGHTBLUE`	Light blue color

LIGHTGREEN	Light green color
LIGHTCYAN	Light cyan color
LIGHTRED	Light red color
LIGHTMAGENTA	Light magenta color
YELLOW	Yellow color
WHITE	White color

Example:

```
setcolor(RED);
circle(200, 200, 50);
```

This draws a red circle.

Other Common Graphics Constants

Some commonly used constants include:

```
SOLID_LINE
DOTTED_LINE
CENTER_LINE
DASHED_LINE
USERBIT_LINE
```

These are used to specify different line styles.

Similarly, fill patterns include:

```
EMPTY_FILL
SOLID_FILL
LINE_FILL
LTSLASH_FILL
SLASH_FILL
BKSLASH_FILL
```

Data Types Used in Graphics

Graphics programming mainly uses the normal C data types along with graphics-specific constants.

Basic Data Types

Data Type	Use
int	Coordinates, sizes, colors, modes
float	Calculations involving decimal values
double	More precise mathematical calculations
char	Characters and text
void	Functions that do not return a value

For example:

```
int x, y, radius;
```

```
float angle;
char ch;
```

Coordinates of graphical objects are generally represented using integer values:

```
int x = 200;
int y = 150;
```

Here, `(200, 150)` represents a position on the graphics screen.

Global Variables Used in Graphics

A **global variable** is a variable declared outside all functions. It can be accessed by multiple functions within the program.

In traditional BGI graphics programming, some predefined global variables/constants may be associated with the graphics environment.

Important graphics-related identifiers include:

`DETECT`

Used to automatically detect the graphics driver.

```
int gd = DETECT;
```

`graphresult()`

Used to determine whether an error occurred during graphics initialization.

```
int errorcode = graphresult();
```

`grapherrormsg()`

Returns a message describing a graphics error.

```
printf("%s", grapherrormsg(errorcode));
```

Example

```
#include <graphics.h>
#include <stdio.h>

int main()
{
    int gd = DETECT, gm;

    initgraph(&gd, &gm, "");

    circle(200, 200, 50);

    getch();
    closegraph();
```

```
    return 0;
}
```

Here:

- `gd` → graphics driver
- `gm` → graphics mode
- `DETECT` → automatically detects the graphics driver
- `initgraph()` → initializes graphics mode

Exam Point: `DETECT` is a predefined constant, not a variable.

Library Functions Used in Drawing

The `graphics.h` library provides several functions for drawing graphical objects.

1. line()

Used to draw a straight line.

Syntax

```
line(x1, y1, x2, y2);
```

Example

```
line(100, 100, 300, 100);
```

It draws a line from `(100,100)` to `(300,100)`.

2. circle()

Used to draw a circle.

Syntax

```
circle(x, y, radius);
```

Example

```
circle(250, 200, 50);
```

Here:

- `(250,200)` → center
- `50` → radius

3. rectangle()

Used to draw a rectangle.

Syntax

```
rectangle(left, top, right, bottom);
```

Example

```
rectangle(100, 100, 300, 200);
```

4. ellipse()

Used to draw an ellipse.

Syntax

```
ellipse(x, y, start_angle, end_angle, x_radius, y_radius);
```

Example

```
ellipse(250, 200, 0, 360, 100, 50);
```

5. arc()

Used to draw an arc.

Syntax

```
arc(x, y, start_angle, end_angle, radius);
```

Example

```
arc(200, 200, 0, 180, 50);
```

This draws the upper half of a circle.

6. putpixel()

Used to draw a single pixel.

Syntax

```
putpixel(x, y, color);
```

Example

```
putpixel(200, 150, RED);
```

7. setcolor()

Used to set the drawing color.

Syntax

```
setcolor(color);
```

Example

```
setcolor(BLUE);
circle(200, 200, 50);
```

Drawing and Filling Images

Graphics programs can create objects and then fill them with different colors or patterns.

`setfillstyle()`

This function specifies the **fill pattern and fill color**.

Syntax

```
setfillstyle(pattern, color);
```

Example

```
setfillstyle(SOLID_FILL, RED);
```

This selects a solid red fill.

`floodfill()`

Used to fill a bounded area with the selected fill pattern.

Syntax

```
floodfill(x, y, boundary_color);
```

Example

```
circle(200, 200, 50);

setfillstyle(SOLID_FILL, BLUE);
floodfill(200, 200, WHITE);
```

The area inside the circle is filled with blue, assuming the circle boundary is white.

`bar()`

Used to draw a filled rectangular bar.

Syntax

```
bar(left, top, right, bottom);
```

Example:

```
setfillstyle(SOLID_FILL, GREEN);
bar(100, 100, 300, 200);
```

`bar3d()`

Used to draw a three-dimensional rectangular bar.

Syntax

```
bar3d(left, top, right, bottom, depth, topflag);
```

Example:

```
bar3d(100, 100, 300, 200, 20, 1);
```

Important Graphics Functions

Some important functions provided by the traditional graphics library are:

Function	Purpose
`initgraph()`	Initializes graphics mode
`closegraph()`	Closes graphics mode
`line()`	Draws a line
`circle()`	Draws a circle
`rectangle()`	Draws a rectangle
`ellipse()`	Draws an ellipse
`arc()`	Draws an arc
`putpixel()`	Draws a pixel
`setcolor()`	Sets drawing color
`setfillstyle()`	Sets filling pattern and color
`floodfill()`	Fills a bounded area
`bar()`	Draws a filled rectangle
`bar3d()`	Draws a 3-D bar
`outtextxy()`	Displays text at a specified position
`cleardevice()`	Clears the graphics screen
`getpixel()`	Returns the color of a pixel

GUI Interaction Within the Program

GUI (Graphical User Interface) is an interface that allows users to interact with a computer program through graphical elements such as:

- Buttons
- Menus
- Windows
- Icons
- Dialog boxes
- Mouse pointers
- Text fields

Instead of entering every command through the keyboard, users can interact with graphical objects.

Mouse Interaction

Graphics programs can also respond to mouse actions.

Common mouse-related operations include:

- Detecting whether the mouse is available
- Showing the mouse pointer
- Getting the mouse position
- Detecting mouse button clicks

In traditional BGI environments, functions such as the following may be used:

```
initmouse();
showmouse();
getmousepos();
```

However, **mouse functions are not part of the standard C language** and their availability depends on the particular graphics library/environment.

Keyboard Interaction

A graphics program can also receive input from the keyboard.

For example:

```
char ch;

ch = getch();

if(ch == 'q')
{
    closegraph();
}
```

Here, the program waits for a key and performs an action depending on the key pressed.

Example: Simple Graphics Program

```
#include <graphics.h>
#include <conio.h>

int main()
{
    int gd = DETECT, gm;

    initgraph(&gd, &gm, "");

    setcolor(RED);
    circle(300, 200, 80);

    setfillstyle(SOLID_FILL, YELLOW);
```

```
    floodfill(300, 200, RED);

    setcolor(BLUE);
    rectangle(200, 300, 400, 400);

    getch();

    closegraph();

    return 0;
}
```

Explanation

1. `initgraph()` starts graphics mode.
2. `setcolor(RED)` selects red for the circle boundary.
3. `circle()` draws a circle.
4. `setfillstyle()` selects yellow as the filling color.
5. `floodfill()` fills the circle.
6. `rectangle()` draws a rectangle.
7. `getch()` waits for a keyboard input.
8. `closegraph()` closes graphics mode.